

InvoiceDoc v2

ระบบจัดการใบแจ้งหนี้ — เข้าใจโครงสร้าง Full-Stack

React · Express · PostgreSQL · Docker

สำหรับผู้ที่ยังไม่เคยเขียนโค้ด

ระบบนี้คืออะไร?

What is InvoiceDoc v2?



จัดการใบแจ้งหนี้

สร้าง / ดู / แก้ไข / พิมพ์
ใบแจ้งหนี้ได้ครบวงจร
รองรับ PDF Export



ข้อมูลพื้นฐาน

จัดการข้อมูล
ลูกค้า (Customer)
และสินค้า (Product)

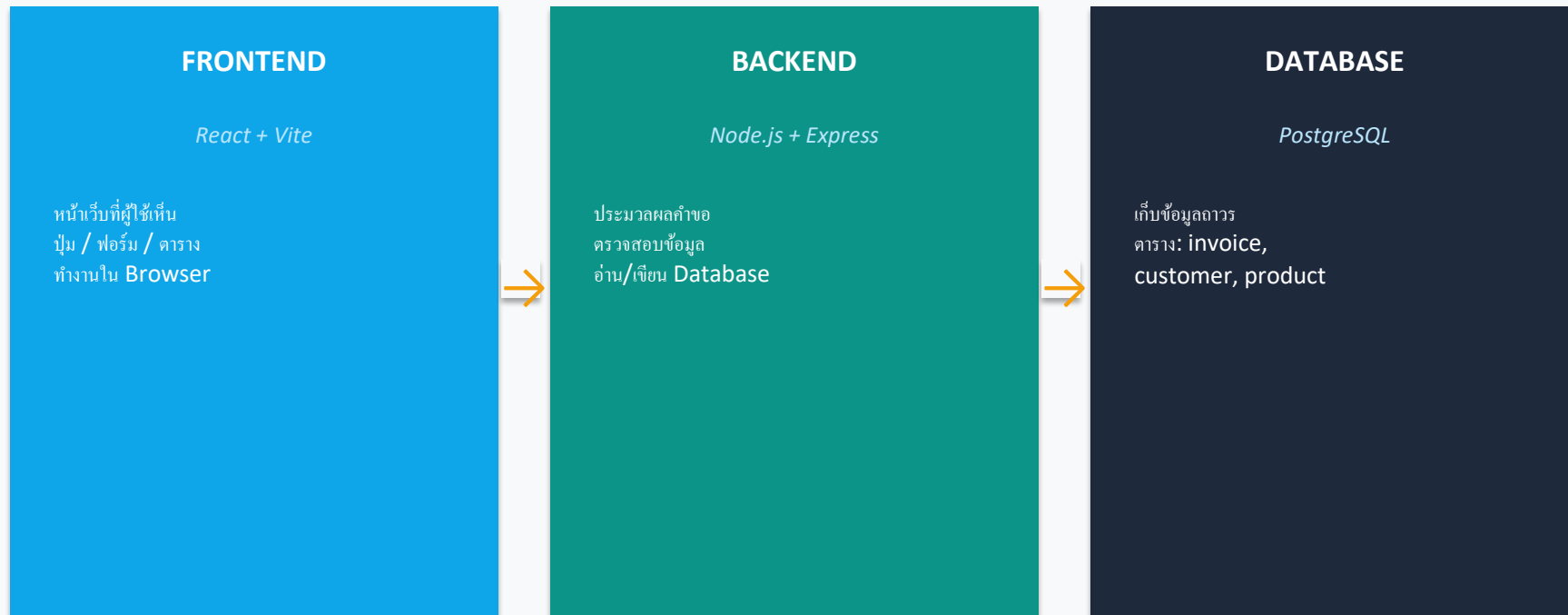


รายงานสรุป

ดูยอดขายตามสินค้า
ยอดขายรายเดือน
ยอดซื้อตามลูกค้า

สถาปัตยกรรม 3 ชั้น

3-Tier Architecture — Frontend → Backend → Database



💡 เปรียบได้กับร้านอาหาร: หน้าร้าน (Frontend) → พนักงาน (Backend) → คลังอาหาร (Database)

เทคโนโลยีที่ใช้

Tech Stack

React 18

Frontend UI Library

สร้างหน้าเว็บแบบ Component
อัปเดตหน้าจออัตโนมัติ

Vite 5

Build Tool (Frontend)

ทำให้ develop เร็วขึ้นมาก
รัน dev server ที่ localhost:5173

Node.js + Express

Backend Framework

รับ HTTP requests
ทำงานเป็น API Server

PostgreSQL 17

Relational Database

เก็บข้อมูลเป็นตาราง
รองรับ SQL queries

Docker

Container Platform

รัน PostgreSQL ใน container
ไม่ต้องติดตั้งเองทั้งหมด

Zod

Validation Library

ตรวจสอบข้อมูลที่ส่งเข้ามา
ก่อนบันทึกลง database

โครงสร้างฐานข้อมูล

Database Schema — ตารางหลักและความสัมพันธ์

country

id
code
name

customer

id
code
name
country_id →
credit_limit

invoice

id
invoice_no
invoice_date
customer_id →
total_amount
vat

units

id
code
name

product

id
code
name
units_id →
unit_price

invoice_line_item

id
invoice_id →
product_id →
quantity
unit_price

→ FK (Foreign Key)
ลิงก์ระหว่างตาราง

โค้ด: Database Schema (SQL)

database/sql/001_schema.sql — สร้างตารางในฐานข้อมูล

SQL

```
CREATE TABLE customer (  
  id          bigint PRIMARY KEY,  
  code        text UNIQUE NOT NULL,  
  name        text NOT NULL,  
  country_id  bigint REFERENCES country(id),  
  credit_limit numeric(12,2)  
);  
  
CREATE TABLE invoice (  
  id          bigint PRIMARY KEY,  
  invoice_no   text UNIQUE NOT NULL,  
  invoice_date date,  
  customer_id  bigint REFERENCES customer(id),  
  total_amount numeric(12,2),  
  amount_due   numeric(12,2)  
);
```



สิ่งที่ควรรู้

REFERENCES = Foreign Key
เชื่อมความสัมพันธ์ระหว่างตาราง

ตัวอย่าง: invoice มี customer_id
อ้างอิงแถวใน customer



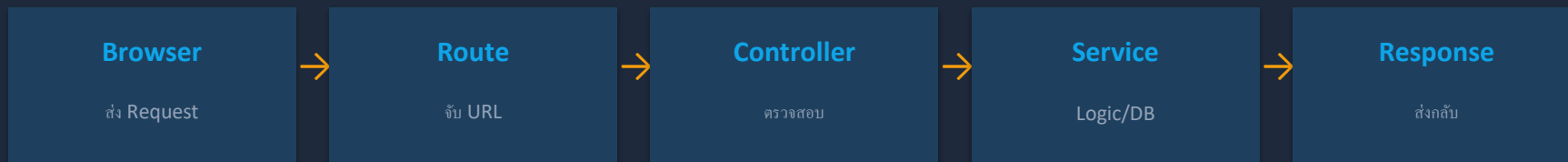
UNIQUE NOT NULL

UNIQUE = ห้ามซ้ำในตาราง
NOT NULL = ต้องมีค่าเสมอ

ตัวอย่าง: invoice_no จะซ้ำไม่ได้
เช่น INV-001, INV-002, ...

Express.js คืออะไร?

Express.js = ตัวกลางรับ HTTP Request แล้วตอบ HTTP Response



ทุก *Request* ที่เข้ามาจะผ่านขั้นตอนนี้เสมอ — ไม่มีการข้าม!

ตัวอย่าง: GET /api/invoices → Route → Controller → Service → Query DB → JSON Response

โค้ด: Routes

server/src/routes/invoices.routes.js — กำหนด URL ของ API

Routes

```
// Invoice API routes
import { Router } from "express";
import * as c from "../controllers/invoices.controller.js";

const r = Router();

r.get("/", c.listInvoices);
r.get("/:invoiceNo", c.getInvoice);
r.post("/", c.createInvoice);
r.put("/:invoiceNo", c.updateInvoice);
r.delete("/:invoiceNo", c.deleteInvoice);

export default r;
```

HTTP Methods

GET

ดึงข้อมูล (อ่าน)

POST

สร้างข้อมูลใหม่

PUT

แก้ไขข้อมูล

DELETE

ลบข้อมูล

/:invoiceNo = Dynamic segment
รับค่า URL เช่น /INV-001

โค้ด: Controller

server/src/controllers/invoices.controller.js — รับ Request แล้วส่งต่อ

Controller

```
// รับ request, เรียก service, ส่ง response
export async function listInvoices(req, res) {
  try {
    const result = await invoicesService.listInvoices(req.query);
    sendList(res, result);
  } catch (err) {
    sendError(res, err?.message, 500);
  }
}

export async function createInvoice(req, res) {
  // Zod ตรวจสอบ body ก่อนเสมอ
  const parsed = CreateInvoiceSchema.safeParse(req.body);
  if (!parsed.success) return sendError(res, 'Validation failed', 400);
  const result = await invoicesService.createInvoice(parsed.data);
  sendCreated(res, result);
}
```

Zod Validation

ตรวจสอบข้อมูลจาก user
ก่อนส่งเข้า database

ถ้าข้อมูลไม่ถูก → ส่ง error 400
ถ้าถูก → บันทึกข้อมูลได้

try / catch

try = ลองรันโค้ด
catch = ถ้าเกิด error จะมาที่นี้

ป้องกันโปรแกรม crash
ส่ง error message กลับไปแทน

โค้ด: Service + SQL Query

server/src/services/invoices.service.js — Business Logic และ Database

Service

```
export async function listInvoices({ page=1, limit=10, search='' }) {
  const offset = (page - 1) * limit;
  const searchParam = `%${search}%`;

  // นับจำนวนทั้งหมด (สำหรับ pagination)
  const countResult = await pool.query(
    `SELECT COUNT(*) FROM invoice i
     JOIN customer c ON c.id = i.customer_id
     WHERE i.invoice_no ILIKE $1 OR c.name ILIKE $1`,
    [searchParam]);

  // ดึงข้อมูลหน้าปัจจุบัน (pagination)
  const { rows } = await pool.query(
    `SELECT i.invoice_no, i.invoice_date, c.name as customer_name
     FROM invoice i JOIN customer c ON c.id = i.customer_id
     ORDER BY invoice_date DESC LIMIT $2 OFFSET $3`,
    [searchParam, limit, offset]);
  return { data: rows, total, page, totalPages };
}
```

✦ Pagination คืออะไร?

แสดงข้อมูลที่ละหน้า
LIMIT = จำนวนต่อหน้า
OFFSET = ข้ามข้อมูลกี่แถว

✦ ILIKE คืออะไร?

LIKE ที่ไม่แคส a/A
\$1 = parameterized query
(ป้องกัน SQL Injection)

✦ JOIN

รวมข้อมูลจาก 2 ตาราง
display ชื่อลูกค้าพร้อม invoice

React คืออะไร?

React = Library สำหรับสร้าง UI แบบ Component-based

<Sidebar>

เมนูนำทาง
Invoices, Customers,
Products, Reports

<InvoiceList>

ตารางรายการ
Invoice พร้อม
ค้นหา + Pagination

<InvoiceForm>

ฟอร์มสร้าง/แก้ไข
Invoice พร้อม
รายการสินค้า

<ReportTable>

ตารางรายงาน
สรุปยอดขาย
พร้อม Filter

แต่ละ Component = ชิ้นส่วน UI ที่ใช้ซ้ำได้ | State เปลี่ยน → UI อัปเดตอัตโนมัติ

โค้ด: Routing (React Router)

client/src/main.jsx — กำหนดว่า URL ไหน แสดงหน้าอะไร

Frontend

```
// URL path → Component ที่จะแสดง
<BrowserRouter>
  <Routes>
    <Route path="/" element={<Navigate to="/invoices"/>} />

    <Route path="/invoices" element={<InvoiceList />} />
    <Route path="/invoices/new" element={<InvoicePage mode="create" />} />
    <Route path="/invoices/:id" element={<InvoicePage mode="view" />} />
    <Route path="/invoices/:id/edit" element={<InvoicePage mode="edit" />} />

    <Route path="/customers" element={<CustomerList />} />
    <Route path="/products" element={<ProductList />} />
    <Route path="/reports/product-sales" element={<Reports />} />
  </Routes>
</BrowserRouter>
```

URL Patterns

<code>/invoices</code>	รายการ invoices
<code>/invoices/new</code>	สร้างใหม่
<code>/invoices/:id</code>	ดูรายละเอียด
<code>/invoices/:id/edit</code>	แก้ไข
<code>/customers</code>	รายการลูกค้า
<code>/reports/...</code>	หน้ารายงาน

โค้ด: API Layer (Frontend)

client/src/api/ — วิธีที่ Frontend เรียก Backend

API Layer

```
// http.js - wrapper สำหรับ fetch
const API_BASE = import.meta.env.VITE_API_BASE || 'http://localhost:4000';

export async function http(path, options={}) {
  const res = await fetch(`${API_BASE}${path}`, options);
  if (!res.ok) throw new Error(await res.text());
  return res.json();
}

// invoices.api.js - ใช้ http() ส่ง request ไป backend
export const getInvoices = (params) =>
  http('/api/invoices?' + new URLSearchParams(params));

export const createInvoice = (data) =>
  http('/api/invoices', { method: 'POST', body: JSON.stringify(data) });

export const deleteInvoice = (id) =>
  http(`/api/invoices/${id}`, { method: 'DELETE' });
```

Fetch API

fetch() = เรียก HTTP Request
ใน JavaScript

React ใช้ fetch เรียก Express
แล้วได้ JSON กลับมา
นำไปแสดงผลบนหน้าเว็บ

VITE_API_BASE

เก็บ URL ของ Backend
อ่านจาก .env file

Dev: http://localhost:4000
Prod: URL จริงบน server

จาก User คลิก ถึง Database



ทุกครั้งที่ user กด Save — ทั้ง 7 ขั้นตอนนี้เกิดขึ้นในเวลาไม่กี่มิลลิวินาที

โค้ด: Transaction (ความปลอดภัยของข้อมูล)

server/src/services/invoices.service.js — BEGIN / COMMIT / ROLLBACK

Service

```
export async function createInvoice(data) {
  const client = await pool.connect();
  try {
    await client.query("begin"); // เริ่ม transaction

    // 1. ค้นหา customer
    const cust = await client.query('SELECT id FROM customer WHERE code=$1', [code]);

    // 2. สร้าง Invoice header
    const inv = await client.query('INSERT INTO invoice (...) VALUES (...)', [...]);

    // 3. สร้าง Line Items ทีละรายการ
    for (const li of line_items) {
      await client.query('INSERT INTO invoice_line_item (...)', [...]);
    }

    await client.query("commit"); // บันทึกทั้งหมดพร้อมกัน
  } catch (err) {
    await client.query("rollback"); // ยกเลิกทั้งหมดถ้า error
    throw err;
  }
}
```

Transaction คืออะไร?

การรันหลาย SQL commands
ให้สำเร็จพร้อมกันทั้งหมด
หรือล้มเหลวทั้งหมด

ตัวอย่าง: ถ้าสร้าง Invoice สำเร็จ
แต่ Line Item ล้มเหลว
→ ROLLBACK ยกเลิก Invoice ด้วย
→ ไม่มีข้อมูลค้างในระบบ

เปรียบเหมือน: "ทุกอย่างสำเร็จ
หรือไม่มีอะไรเกิดขึ้นเลย"

สรุป

ภาพรวมของ InvoiceDoc v2 Full-Stack Application

Database

SQL Schema สร้างตาราง 6 ตาราง
เชื่อมกันด้วย Foreign Keys

Backend

Express: Route → Controller → Service
ทำ CRUD + SQL Query + Transaction

Frontend

React: Component + Router
เรียก Backend ผ่าน fetch API

Security

Zod Validation ตรวจสอบ input
Parameterized Query ป้องกัน SQL Injection

 [ขั้นต่อไป: ลองแก้ไขโค้ด เพิ่มฟีเจอร์ใหม่ หรือดูเอกสาร GUIDE.md ในโปรเจกต์](#)